

Advance Playwright

Week-05 Gate-05

Purpose of this document. A reference demo report that covers *every* front the gate asks for — setup, context, framework, scenarios, exact commands, results, evidence

- **Participant:** Monisha Puttaswamy **Date:** 04-07-2026
- **Assessment:** Advance playwright UI testing on sdet-retail-app.

1. Objective & user story

Automate the end-to-end retail shopping journey on a nykaa ,run it headless on stable explicit waits, and prove it with playwright results.

2. Environment & setup

Layer	Pinned choice
-------	---------------

Language	ts
Browser	chrome
Test platform	chromium
Target app	Nykaa machine
windows	

Prerequisites

- 1) Node.js
- 2) npm / npx

3. Framework design

1. **Artifact Management (artifact-management.spec.ts)**
2. **Artifact Failure Demo (artifact-failure.spec.ts)**
3. **Diagnostics and Logging (diagnostics-logging.spec.ts)**

4. **Flaky Test Demonstration (makeitflaky.spec.ts)**
5. **Offline & Network Resilience (network-offline.spec.ts)**
6. **Refund Management (refund.spec.ts)**
7. **Logging Utility (logger.ts)**
8. **Diagnostic Fixture (diagnostic.fixture.ts)**
9. **App Fixture (app.fixture.ts)**
10. **Refund API Helper (refund-return.api.ts)**

Project Overview

This project implements an advanced Playwright automation framework that combines **UI Automation** and **API Automation** using custom fixtures, structured logging, evidence collection, artifact management, and reusable API utilities. The framework validates multiple real-world business scenarios while maintaining detailed reporting, diagnostics, and traceability.

The **Diagnostics and Logging** module verifies the complete checkout journey by generating a unique correlation ID for each test, capturing structured logs, validating cart and order APIs, and ensuring sensitive information such as authorization tokens and payment details are masked before logging. The **Artifact Management** module captures API responses as execution evidence and automatically attaches them to the Playwright report, while the **Artifact Failure Demo** demonstrates Playwright's automatic generation of screenshots, traces, videos, and failure artifacts whenever a test fails.

The project also includes **Flaky Test Demonstration** scenarios to showcase common causes of unstable UI automation, such as asynchronous timing issues and brittle position-based locators, highlighting the importance of proper synchronization and stable locator strategies. The **Offline and Network Resilience** module validates application behavior during network interruptions by simulating offline and online modes, verifying offline and online banners, queuing sales while offline, automatically synchronizing queued data after reconnection, handling optimistic UI updates, rolling back failed operations, implementing retry mechanisms, and ensuring exactly-once delivery through idempotency.

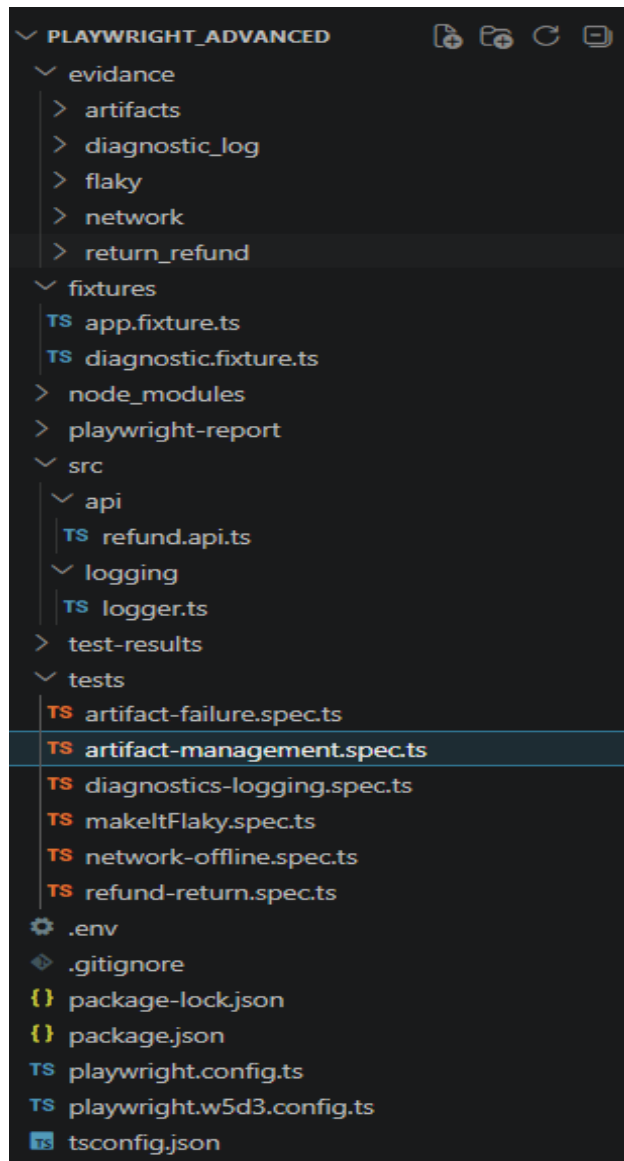
The **Return and Refund** module validates the complete refund workflow by creating test orders, processing partial and full refunds, checking refund eligibility, preventing over-refunds and duplicate refunds, verifying idempotent refund processing, validating ledger updates, ensuring accurate money calculations using paise values, and confirming that the UI correctly reflects refund status and refund totals. It also verifies tax calculations, tax distribution, refundable balances, refund status transitions, and overall financial accuracy.

To support these tests, the framework includes reusable components such as a **custom logging utility** that generates structured logs, formats log messages, adds timestamps and metadata, and automatically redacts sensitive information before writing logs. The **diagnostic fixture** automatically generates correlation IDs, injects them into HTTP requests, captures execution logs, and attaches diagnostic logs to failed test reports. The **application fixture** provides automatic evidence collection by storing JSON and text-based evidence and attaching them to Playwright reports after execution. A reusable **Refund API Helper** centralizes

common API operations such as order creation, refund processing, refund validation, ledger retrieval, returns page navigation, and idempotency key generation, improving code reusability and maintainability.

Overall, the project demonstrates advanced Playwright capabilities including **UI and API automation, hybrid testing, custom fixtures, structured logging, evidence collection, artifact management, diagnostics, network simulation, route interception, offline testing, retry validation, flaky test demonstration, correlation ID tracking, idempotency testing, refund validation, money precision verification, negative testing, data-driven testing, environment-based configuration, Playwright HTML reporting, Trace Viewer integration, automatic log attachments, and JSON evidence attachments**, resulting in a scalable, maintainable, and production-ready test automation framework.

4. Project structure



5. Scenarios covered

Validated **artifact management**, **diagnostic logging**, **evidence collection**, and **flaky test detection** to improve debugging and test reliability.

Tested **network resilience** by simulating offline/online modes, queued sales synchronization, optimistic UI rollback, retry logic, and idempotent request handling.

Verified the **return and refund workflow**, including partial and full refunds, refund eligibility, rejected and duplicate refunds, money precision, ledger validation, and end-to-end UI/API integration.

6. How to run

To install Playwright

npm init playwright@latest

To Run Playwright Test

npx playwright test

To run specific test file

npx playwright test tests/example.spec.ts

To run by specific test name

npx playwright test -g "test name"

To run test headed mode npx

playwright test --headed

To run test in UI mode

npx playwright test --ui

To run test in debug mode npx

playwright test --debug

To run parallel tests

npx playwright test --workers=4

To trace the test

npx playwright test --trace on

To view report

npx playwright show-report

7. Flow under the test

Artifact Management

Open Application → Perform Action → Capture Evidence → Attach Artifacts

Diagnostics & Logging

Start Test → Generate Correlation ID → Execute API Flow → Record Logs → Validate Responses

Flaky Testing

Execute Unstable Scenario → Observe Timing/Locator Issues → Capture Failure Details

Network Testing

Go Offline → Queue Sales → Restore Network → Sync Data → Verify Retry & Idempotency

Return & Refund

Create Order → Process Refund → Validate Ledger & Money Precision → Verify UI → Prevent Duplicate/Over Refunds

8. Problem faced

```
Retry #2 -----
Error: expect(received).toBe(expected) // Object.is equality

Expected: 201
Received: 409

44 |
45 |     // Verify the product was added successfully
> 46 |     expect(addItem.status()).toBe(201);
    |                                     ^
47 |     // Verify the cart retrieval was successful
48 |     expect(cart.status()).toBe(200);
49 |
    at C:\Users\299174\Desktop\SDET\week05gate05\Playwright_Advanced\tests\artifact-management.spec.ts:46:32

attachment #1: screenshot (image/png) -----
test-results\artifact-management-Week05-ccb1a-aching-it-on-a-passing-test-chromium-retry2-repeat48\test-failed-1.png

attachment #2: video (video/webm) -----
test-results\artifact-management-Week05-ccb1a-aching-it-on-a-passing-test-chromium-retry2-repeat48\video.webm

Error Context: test-results\artifact-management-Week05-ccb1a-aching-it-on-a-passing-test-chromium-retry2-repeat48\error-context.md

Retry #3 -----
Error: expect(received).toBe(expected) // Object.is equality

Expected: 201
Received: 409
```

While trying to run this I was frequently getting failed and some times it was flaky it took time to understand the issue. The issue was we can run only 5 times in a active backend the solution which I got is rerun the backend and then try to test. It was passing issue got solved

8.1 Playwright Report

view report

npx playwright show-report

All 14

✓ Passed 11

✗ Failed 1

⚠ Flaky 2

Skipped 0

⚙

⚙

Project: chromium

4/7/2026, 5:10:41 pm Total time: 51.2s

▼ artifact-failure.spec.ts

✗ Week05 Gate05 - Attaching Artifacts on Failure - Test to check the artifacts on failure

chromium

32.5s

artifact-failure.spec.ts:5

View Trace

▼ artifact-management.spec.ts

⚠ Week05 Gate05 - Artifact Management - captures cart evidence without attaching it on a passing test

chromium

9.0s

artifact-management.spec.ts:14

View Trace

▼ makeItFlaky.spec.ts

⚠ Flaky Test Demonstration - bad example: assert cart total before the async request has settled

chromium

7.6s

makeItFlaky.spec.ts:5

View Trace

✓ Flaky Test Demonstration - bad example: brittle locator depends on element position

chromium

2.6s

makeItFlaky.spec.ts:35

▼ diagnostics-logging.spec.ts

✓ Week05 Gate05 Diagnostics and Logging - records a correlated checkout diagnostic trail

chromium

3.2s

diagnostics-logging.spec.ts:13

✓ Week05 Gate05 Diagnostics and Logging - redacts sensitive fields in structured logs

chromium

2.6s

diagnostics-logging.spec.ts:109

▼ network-offline.spec.ts

✓ Making Network Offline and Online - Prove the offline banner

chromium

5.4s

network-offline.spec.ts:5

✓ Making Network Offline and Online - Rollback a failed optimistic write

chromium

2.7s

network-offline.spec.ts:64

✓ Making Network Offline and Online - sync once, even through a flaky reconnect

chromium

3.4s

network-offline.spec.ts:105

▼ refund-return.spec.ts

✓ Create test order and check Return and Refund along with the trace view - Refund functionality for partial refund

chromium

2.6s

refund-return.spec.ts:16

✓ Create test order and check Return and Refund along with the trace view - Refund functionality for full refund

chromium

2.5s

refund-return.spec.ts:69

✓ Create test order and check Return and Refund along with the trace view - Valid refund

chromium

2.0s

refund-return.spec.ts:131

✓ Create test order and check Return and Refund along with the trace view - Over refund

chromium

2.3s

refund-return.spec.ts:131

✓ Create test order and check Return and Refund along with the trace view - Rejected Refund, Duplicate refund, Idempotency

chromium

1.7s

refund-return.spec.ts:158

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
chrome-headless-shell + - [x]

C:\Users\299174\Desktop\SDet\week05gate05\Playwright_Advanced>npx playwright test refund-return.spec.ts --repeat-each=50

Running 250 tests using 4 workers
[chromium] > tests\refund-return.spec.ts:131:9 > Create test order and check Return and Refund along with the trace view > Valid refund
[INFO] 2026-07-04T12:01:11.803Z - test started
[INFO] 2026-07-04T12:01:11.806Z - Creating order
[INFO] 2026-07-04T12:01:11.937Z - Checking refund eligibility
[INFO] 2026-07-04T12:01:11.976Z - Refund verdict {"verdict":"APPROVED"}
[INFO] 2026-07-04T12:01:11.986Z - test finished {"status":"passed"}
[chromium] > tests\refund-return.spec.ts:131:9 > Create test order and check Return and Refund along with the trace view > Over refund
[INFO] 2026-07-04T12:01:12.021Z - test started
[INFO] 2026-07-04T12:01:12.024Z - Creating order
[chromium] > tests\refund-return.spec.ts:16:9 > Create test order and check Return and Refund along with the trace view > Refund functionality for partial refund
[INFO] 2026-07-04T12:01:12.102Z - test started
[chromium] > tests\refund-return.spec.ts:131:9 > Create test order and check Return and Refund along with the trace view > Over refund
[INFO] 2026-07-04T12:01:12.111Z - Checking refund eligibility
[chromium] > tests\refund-return.spec.ts:16:9 > Create test order and check Return and Refund along with the trace view > Refund functionality for partial refund
[INFO] 2026-07-04T12:01:12.106Z - Seeding order
[chromium] > tests\refund-return.spec.ts:131:9 > Create test order and check Return and Refund along with the trace view > Over refund
[INFO] 2026-07-04T12:01:12.119Z - Refund verdict {"verdict":"OVER_REFUND"}
[INFO] 2026-07-04T12:01:12.130Z - test finished {"status":"passed"}
[chromium] > tests\refund-return.spec.ts:16:9 > Create test order and check Return and Refund along with the trace view > Refund functionality for partial refund
[INFO] 2026-07-04T12:01:12.190Z - Opening Returns page {"orderId":"7003"}
[chromium] > tests\refund-return.spec.ts:69:9 > Create test order and check Return and Refund along with the trace view > Refund functionality for full refund
[INFO] 2026-07-04T12:01:12.356Z - test started
[INFO] 2026-07-04T12:01:12.359Z - Creating order
[INFO] 2026-07-04T12:01:12.412Z - Refunding entire order
[INFO] 2026-07-04T12:01:12.431Z - Full refund completed {"status":"REFUNDED"}
[chromium] > tests\refund-return.spec.ts:16:9 > Create test order and check Return and Refund along with the trace view > Refund functionality for partial refund
[INFO] 2026-07-04T12:01:12.799Z - Submitting partial refund
[INFO] 2026-07-04T12:01:12.825Z - Ledger verified {"status":"PARTIALLY_REFUNDED","refundCount":1}
[INFO] 2026-07-04T12:01:12.947Z - test finished {"status":"passed"}
[chromium] > tests\refund-return.spec.ts:69:9 > Create test order and check Return and Refund along with the trace view > Refund functionality for full refund

```

Running it 50 times

Result

9. Environment details

BaseUrl=http://localhost:5173

Browser=Chrome (headless)

Build=local

OS= windows 11

Java=22 Playwright= 1.61.1

10. Config files

```
import { defineConfig, devices } from "@playwright/test";

export default defineConfig({
  testDir: "./tests",
  timeout: 30_000,
  fullyParallel: true,
  workers: process.env.CI ? 4 : undefined,
  retries: process.env.CI ? 2 : 3,
  reporter: process.env.CI ? [
    ["list"],
    ["blob"]
  ] : [
    ["html"]
  ],
  use: {
    baseURL: process.env.BASE_URL || "http://localhost:5173",
    trace: "on-first-retry",
    screenshot: "only-on-failure",
    video: "retain-on-failure"
  },
  projects: [
    {
      name: "chromium",
      use: { ...devices["Desktop Chrome"] }
    }
  ]
});
```

```
ts playwright.w5d3.config.ts > default
1 import { defineConfig, devices } from "@playwright/test";
2 import process from "node:process";
3
4 const CI = !!process.env.CI;
5
6 export default defineConfig({
7   testDir: "./tests",
8   timeout: 30_000,
9   fullyParallel: true,
10  retries: CI ? 2 : 0,
11
12  reporter: CI
13    ? [
        ["list"],
        ["blob"],
        ["html", { open: "never" }]
      ] : [
        ["html"]
      ],
14
15  use: {
16    baseURL: process.env.BASE_URL || "http://localhost:5173",
17    screenshot: CI ? "only-on-failure" : "off",
18    trace: CI ? "on-first-retry" : "off",
19    video: CI ? "retain-on-failure" : "off",
20  },
21
22  projects: [
23    {
24      name: "chromium",
25      use: {
26        ...devices["Desktop Chrome"],
27      },
28    },
29  ],
30
31  --
```

11. Conclusion

This project successfully demonstrates an advanced Playwright automation framework by combining **UI and API testing** with custom fixtures, structured logging, artifact management,

and evidence collection. It validates critical business scenarios such as diagnostics, network resilience, flaky test handling, and the complete return and refund workflow, including idempotency and money precision.

The framework provides reliable, maintainable, and scalable test automation with comprehensive reporting, making it easier to identify failures, debug issues, and ensure application quality across different test scenarios.
